

Artificial Intelligence

AI2002

Rushda Muneer

Spring 2026

Algorithm A - Admissibility

- Consider the evaluation function $f(n) = g(n) + h(n)$, where
 - n is any state encountered in the search
 - $g(n)$ is the cost of n from start state
 - $h(n)$ is the heuristic estimate of the cost of going from n to the goal
- If this evaluation function is used with the best-first search algorithm, the result is called **algorithm A**
- **A search algorithm is admissible if, for any graph, it always terminates in the optimal solution path whenever a path from the start to a goal exists**

Algorithm A for 8 puzzle

- Both state e and f have same $f(n)=5$
- State e is examined first and produces children h and i
- Although the number of tiles out of place for h and f are the same (heuristics), the depth measure taken as path cost $g(n)$ determines the direction of search
- Therefore the algorithm picks the shallower state f to explore next over the deeper state h
- The $g(n)$ component of the evaluation function gives the search a breath-first flavor
- This prevents it from being misled by an erroneous evaluation: if the heuristic continuously returns “good” evaluations for all states in a path that fails to reach a goal
- In algorithm A, $g(n)$ will continue to dominate $h(n)$ and force the search back to a shorter path evaluation.

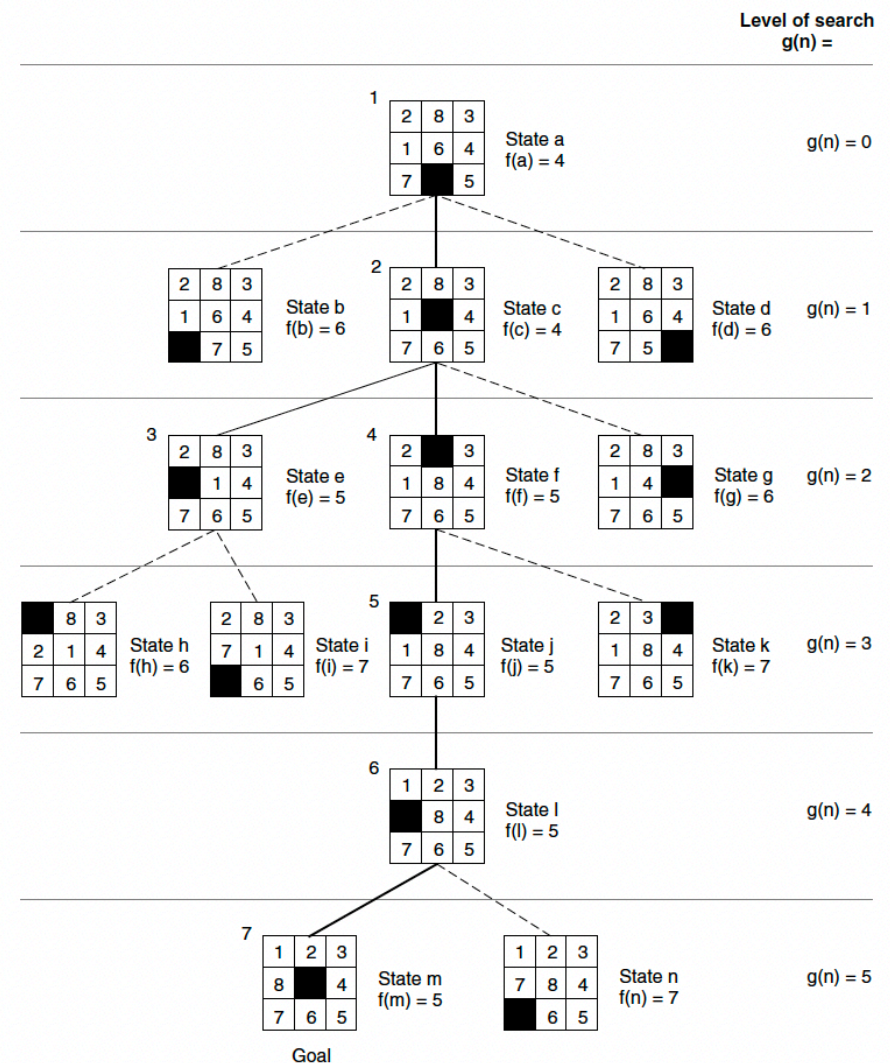


Figure 4.16 State space generated in heuristic search of the 8-puzzle graph.

Algorithm A*

- Consider the evaluation function
 - $f(n) = g(n) + h(n)$
- Now consider another evaluation function
 - $f^*(n) = g^*(n) + h^*(n)$
- Where $g^*(n)$ is the cost of the shortest path from the start node to n and $h^*(n)$ returns the actual cost of the shortest path from n to the goal
- Thus $f^*(n)$ is the actual cost of the optimal path from start node to a goal node that passes through node n

Algorithm A*

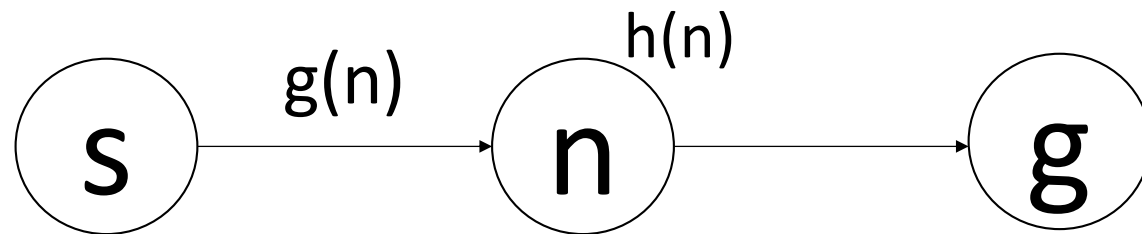
- If we apply best-first search algorithm with the evaluation function $f^*(n)$ the resulting search is admissible.
- If algorithm A is used with an evaluation function in which $h(n)$ is less than or equal to the cost of the minimal path from n to the goal, the resulting search algorithm is called algorithm A*
- It is now possible to state a property of A* algorithms:
 - All A* algorithms are admissible.
- The admissibility of A* algorithms is a theorem.
- The theorem says that any A* algorithm, i.e., one that uses a heuristic $h(n)$ such that $h(n) \leq h^*(n)$ for all n , is guaranteed to find the minimal path from n to the goal, if such a path exists.

Algorithm A* and 8-Puzzle

- Several heuristics from the 8-puzzle provide examples of A* algorithms.
- Although we may not be able to compute the value of $h^*(n)$ for the 8-puzzle, we may determine when a heuristic is bounded from above by the actual cost of the shortest path to a goal.
- For instance, the heuristic of counting the number of tiles not in the goal position is certainly less than or equal to the number of moves required to move them to their goal position. Thus, this heuristic is admissible and guarantees an optimal (or shortest) solution path when that path exists.
- The sum of the direct distances of tiles out of place is also less than or equal to the minimum actual path.
- Using small multipliers for direct tile reversals gives an admissible heuristic

Admissible heuristics

Cost function: $f(n) = g(n) + h(n)$



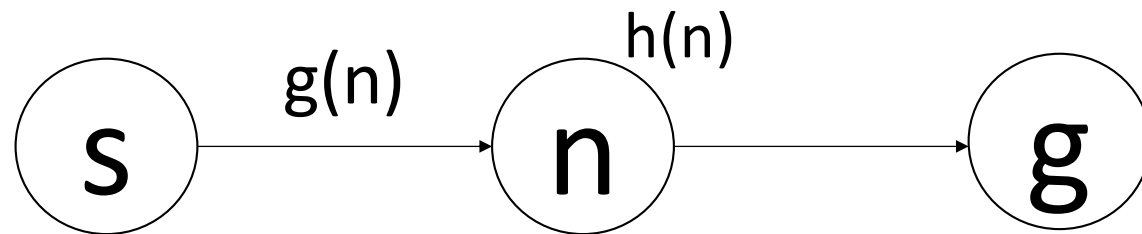
Actual Cost: $f^*(n) = g^*(n) + h^*(n)$

$g^*(n)$ is the cost of the shortest path from the start node to n

$h^*(n)$ returns the actual cost of the shortest path from n to the goal

Admissible heuristics

Cost function: $f(n) = g(n) + h(n)$

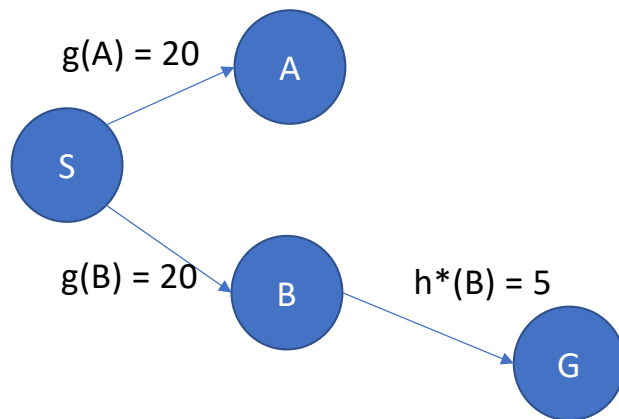
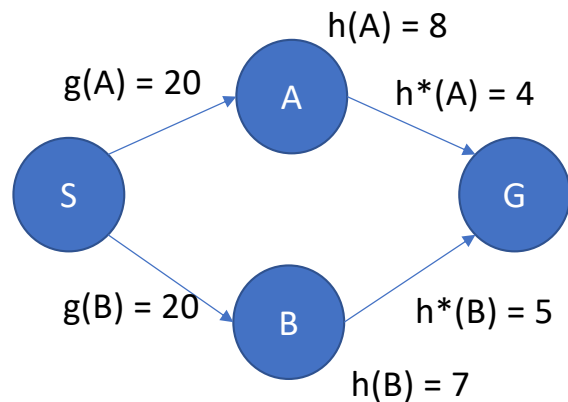


If $h(n) \leq h^*(n) \rightarrow$ underestimation

If $h(n) \geq h^*(n) \rightarrow$ overestimation

A* provides a lower bound on the actual cost, meaning it is always optimistic.

Admissible heuristics



If we overestimate the heuristics,

$h(A) = 8$

$h(B) = 7$

$f(A) = 20 + 8 = 28$

$f(B) = 20 + 7 = 27$

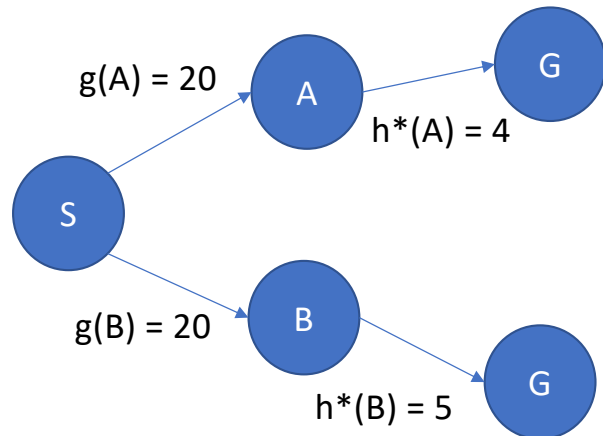
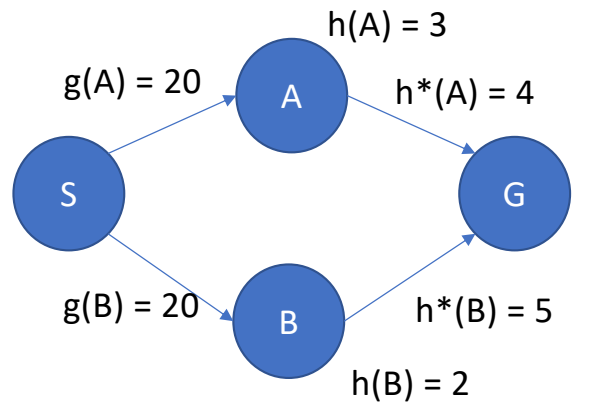
We expand B

We check child G, $f(G) = 25 + 0 = 25$

Since $f(G) \leq f(A)$, in our priority queue we expand G and the search stops

Although there was an optimal path available we couldn't reach it because of the overestimated heuristics

Admissible heuristics



If we underestimate the heuristics,

$h(A) = 3$

$h(B) = 2$

$f(A) = 20 + 3 = 23$

$f(B) = 20 + 2 = 22$

We expand B

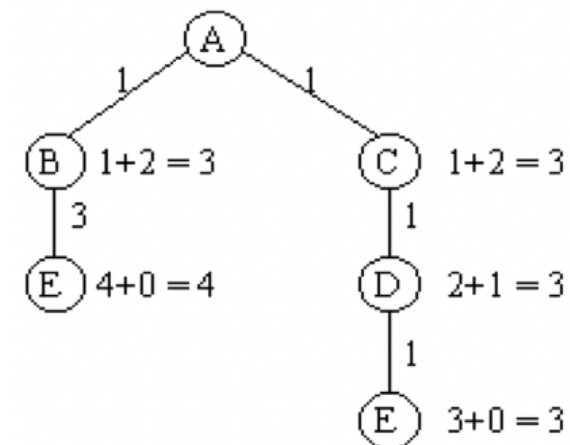
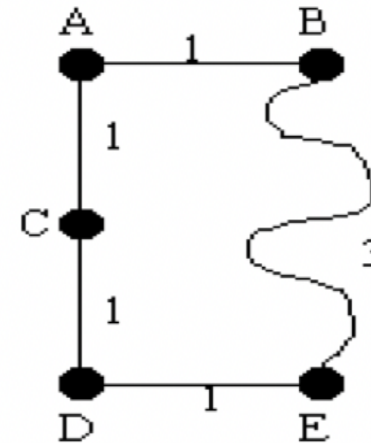
We check child G, $f(G) = 25 + 0 = 25$

Since $f(G) \geq f(A)$, in our priority queue we will expand A first, from A we reach G with $f(G) = 24$

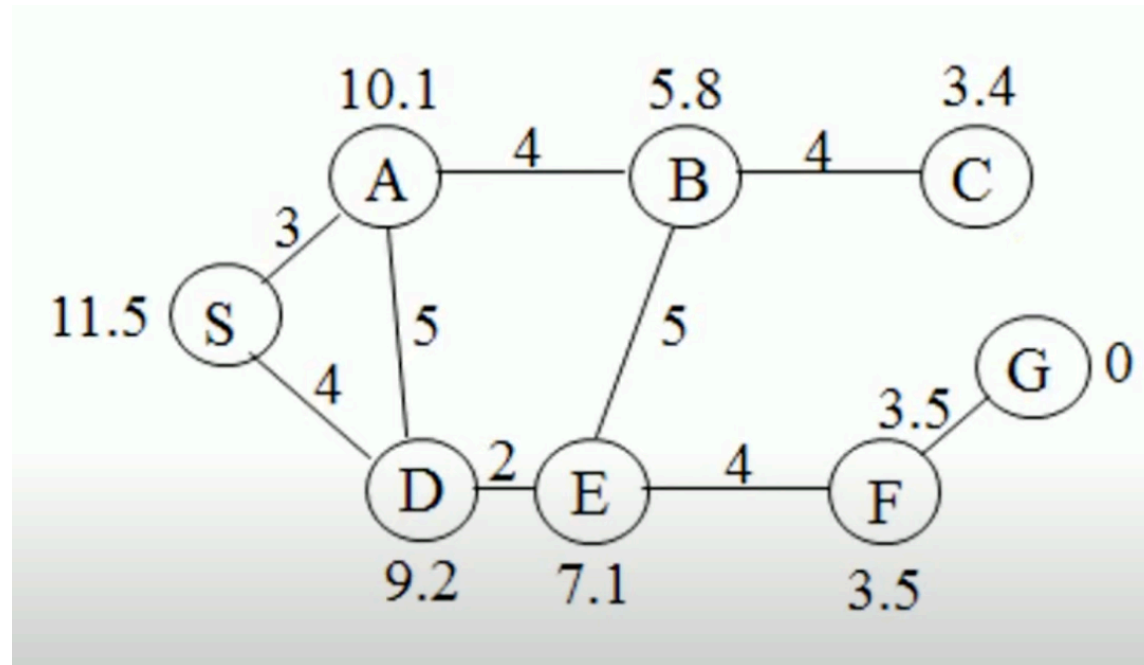
Now the optimal path is found and the search is A^* , (because the heuristics are admissible)

A* Search - Example

- We want to go from A to E
- In the graph we are given the actual cost of moving from one node to another
- From both B and C the cost of going from A is evaluated and added to the estimated distance of going from here to E.
- The heuristics (estimates) from both B and C are 2
- The overall cost for both B and C turns out to be 3
- Let's assume that the search picks leftmost node for evaluation in case of equal values
- We expand node B
- Its child E has an estimate value $4+0 = 4$
- However since estimate for C is 3, we expand it first
- The estimate of it's child D is 3, we will pick D for expansion
- Now E is estimated with the cost of $3+0 = 3$
- This hence gives us an optimal answer for our search
- Here the heuristics were admissible



Activity – Check if the heuristics are admissible?



Activity – Run A* search from S to G

